

PROJECT REPORT

AI-Powered Credit Card Fraud Detection System

A Machine Learning based Web Application for Classifying Fraudulent and Legitimate Credit Card Transactions

Prepared by: Ananya Sharma

Domain: Machine Learning / Financial Data Analytics

Tech Stack: Python, Flask, Scikit-learn, XGBoost, Chart.js

Date: July 2026

Table of Contents

- 1. Introduction 3**
 - 1.1 Background 3
 - 1.2 Problem Statement 3
 - 1.3 Objectives 3
 - 1.4 Scope 3
- 2. Background and Related Work 4**
 - 2.1 Class Imbalance 4
 - 2.2 Feature Anonymisation 4
 - 2.3 Model Choice 4
- 3. Dataset Description 4**
 - 3.1 Source and Composition 4
 - 3.2 Class Distribution 5
 - 3.3 Data Quality 5
- 4. Methodology 5**
 - 4.1 System Architecture 5
 - 4.2 Data Preprocessing Pipeline 5
 - 4.3 Model Training 6
 - 4.4 Evaluation Strategy 6
- 5. System Implementation 7**
 - 5.1 Backend API (Flask) 7
 - 5.2 Command-Line Prediction Utility 7
 - 5.3 Frontend Interface 8
 - 5.4 Deployment Configuration 8
- 6. Results and Evaluation 8**
 - 6.1 Model Comparison 8
 - 6.2 ROC Curve Comparison 9
 - 6.3 Confusion Matrices 9
 - 6.4 Illustrative Evaluation on the Sample Dataset 9
 - 6.5 Discussion 10
- 7. Testing and Validation 10**
 - 7.1 Functional Testing 10
 - 7.2 Model Validation 11
 - 7.3 Known Limitations Observed During Testing 11
- 8. Project Structure and Technology Stack 11**
 - 8.1 Technology Stack 11
 - 8.2 Project Folder Structure 11
- 9. Conclusion and Future Work 12**
 - 9.1 Conclusion 12
 - 9.2 Future Improvements 12
- 10. References 12**

1. Introduction

1.1 Background

Credit card fraud is a persistent and costly problem for banks, payment processors, and consumers worldwide. As digital and card-not-present transactions have grown, fraudulent activity has grown alongside them, and manual, rule-based fraud checks struggle to keep pace with the volume and evolving patterns of modern transaction data. Machine learning offers a scalable alternative: models can be trained on historical transaction records to automatically recognise the subtle statistical patterns that separate genuine purchases from fraudulent ones, and can then screen new transactions in real time or in batch.

This report documents the design, implementation, and evaluation of an end-to-end Credit Card Fraud Detection System. The system combines a trained ensemble of classification models with a Flask-based backend API and a browser-based frontend, allowing a user to upload a CSV file of transactions and instantly receive fraud predictions, summary statistics, and a downloadable prediction report.

1.2 Problem Statement

Given an anonymised dataset of credit card transactions, each described by numerical features derived from a PCA transformation together with the transaction Time and Amount, the goal is to build a binary classifier that labels each transaction as Legitimate (Class = 0) or Fraudulent (Class = 1). The central challenge is that fraud is rare: fraudulent transactions form a very small minority of all transactions, so a naive model that always predicts "legitimate" would already achieve very high accuracy while being completely useless. The system must therefore be designed and evaluated with this class imbalance explicitly in mind.

1.3 Objectives

- Build a data preprocessing pipeline that cleans, scales, and balances the transaction dataset.
- Train and compare multiple classification algorithms — Logistic Regression, Random Forest, and XGBoost — for fraud detection.
- Evaluate each model using metrics appropriate for imbalanced classification (precision, recall, F1-score, ROC-AUC), not accuracy alone.
- Select the best-performing model and package it for deployment.
- Expose the trained model through a REST API built with Flask.
- Build a web interface that lets a non-technical user upload a CSV file and view fraud predictions, charts, and a downloadable report.

1.4 Scope

The project covers the full applied machine learning lifecycle for this task: data understanding, preprocessing, model training, evaluation, and deployment as a lightweight web application. It does not cover real-time transaction streaming, integration with a live payment gateway, or user authentication, all of which are noted as future improvements in Section 9.

2. Background and Related Work

Fraud detection is a well-studied application of anomaly detection and supervised classification. Two characteristics of the problem recur across the literature and directly shaped the design choices in this project:

2.1 Class Imbalance

In real-world card transaction datasets, fraudulent transactions typically account for well under 1% of all records. A classifier trained naively on such data tends to be biased toward the majority class, achieving high overall accuracy while missing most fraud cases. Standard approaches to counter this include resampling the training data (oversampling the minority class, undersampling the majority class, or synthetic oversampling techniques such as SMOTE), using class-weighted loss functions, and evaluating with metrics that are sensitive to minority-class performance rather than raw accuracy.

2.2 Feature Anonymisation

Publicly available card-fraud datasets, including the one used in this project, are commonly released with the majority of features already transformed via Principal Component Analysis (PCA) to protect cardholder privacy. This means the individual features (V1 to V28) do not have direct real-world interpretations; only Time and Amount remain in their original units. This constrains feature engineering but is standard practice for this class of problem.

2.3 Model Choice

Tree-based ensemble methods such as Random Forest and Gradient Boosting (including XGBoost) are widely reported to perform strongly on tabular fraud-detection tasks because they can capture non-linear feature interactions without extensive manual feature engineering, and they are comparatively robust to unscaled or skewed numerical features. Logistic Regression is commonly retained as an interpretable, fast-training baseline against which the more complex models are compared. This project follows that same three-model comparison strategy.

3. Dataset Description

3.1 Source and Composition

The project uses the Credit Card Fraud Detection dataset, a widely used benchmark containing anonymised transactions made by European cardholders. Each record represents a single transaction and is described by 31 columns:

- Time — the number of seconds elapsed between each transaction and the first transaction in the dataset.
- V1 – V28 — twenty-eight numerical features produced by a PCA transformation of the original transaction attributes, included to protect sensitive information.
- Amount — the transaction amount, left in its original scale.
- Class — the target label: 0 for a legitimate transaction, 1 for a fraudulent transaction.

3.2 Class Distribution

A representative sample of the dataset (10,000 transactions) bundled with this project was used to illustrate and validate the pipeline described in this report. Its class balance is summarised below:

Class	Label	Transaction Count	Percentage
0	Legitimate	9,508	95.08%
1	Fraud	492	4.92%
Total	—	10,000	100%

Even in this curated sample, fraudulent transactions remain a small minority of the data. In the full-scale version of this dataset used for model training, the imbalance is far more extreme — fraud typically accounts for well under 1% of all transactions — which is why the preprocessing pipeline (Section 4) applies synthetic oversampling before training.

3.3 Data Quality

- No missing values are present in the numerical feature columns.
- All V1–V28 features are already numeric and PCA-transformed, requiring no additional encoding.
- Time and Amount are on very different scales from the PCA components and from each other, motivating targeted feature scaling.

4. Methodology

4.1 System Architecture

The system is organised into three cooperating layers, mirroring the project's folder structure:

- Data & Model Layer (Dataset/, Models/, src/) — holds the raw data, the trained model artifacts (model, scaler, and feature-column list, serialised with joblib), and the Python scripts used to preprocess data, train models, evaluate them, and run standalone predictions.
- Backend / API Layer (app.py) — a Flask application that loads the trained Random Forest model, scaler, and feature list once at start-up, exposes a /predict endpoint that accepts a CSV upload, and exposes a /download endpoint that returns the resulting predictions as a CSV report.
- Frontend Layer (frontend/, templates/, static/) — an HTML, CSS, and JavaScript interface (built with Bootstrap 5 and Chart.js) that lets a user upload a file, view summary statistics and interactive pie/bar charts, search the prediction table, and download the report.

4.2 Data Preprocessing Pipeline

Preprocessing is implemented in src/preprocess.py and mirrored inside app.py so that new transactions submitted through the web interface are transformed identically to the data the model was trained on. The pipeline performs the following steps:

1. Load the raw CSV into a pandas DataFrame.
2. Separate the feature matrix X (all columns except Class) from the target vector y (the Class column).

3. Split the data into training (80%) and test (20%) sets using a stratified split, so that the proportion of fraud cases is preserved in both partitions.
4. Fit a StandardScaler on the Time and Amount columns of the training set only, and apply it to both the training and test sets. The remaining V1–V28 columns are left unscaled, since they are already the output of a PCA transformation and are approximately standardised.
5. Persist the fitted scaler and the ordered list of feature column names with joblib, so the exact same transformation can be replayed at inference time inside the Flask API.
6. Apply SMOTE (Synthetic Minority Over-sampling Technique) to the training set only, generating synthetic fraud examples until the two classes are balanced. The test set is left untouched so that evaluation reflects real-world class proportions.

```
X = df.drop("Class", axis=1)
y = df["Class"]
X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.20, random_state=42, stratify=y)

scaler = StandardScaler()
X_train[["Time", "Amount"]] = scaler.fit_transform(X_train[["Time", "Amount"]])
X_test[["Time", "Amount"]] = scaler.transform(X_test[["Time", "Amount"]])

smote = SMOTE(random_state=42)
X_train_smote, y_train_smote = smote.fit_resample(X_train, y_train)
```

Code Snippet 1 — Core preprocessing steps (src/preprocess.py)

4.3 Model Training

Three classification algorithms were trained on the SMOTE-balanced training data in `src/train.py`, each chosen to represent a different modelling family:

Model	Type	Key Hyperparameters
Logistic Regression	Linear model	<code>max_iter = 1000</code> , <code>random_state = 42</code>
Random Forest	Bagged decision-tree ensemble	<code>n_estimators = 100</code> , <code>random_state = 42</code> , <code>n_jobs = -1</code>
XGBoost	Gradient-boosted decision-tree ensemble	<code>n_estimators = 150</code> , <code>max_depth = 6</code> , <code>learning_rate = 0.1</code>

Each model was trained on the same SMOTE-resampled training set and evaluated on the same untouched, imbalanced test set, ensuring a fair, like-for-like comparison. Trained models were serialised to the `Models/` directory with joblib for reuse by the evaluation script and the Flask API.

4.4 Evaluation Strategy

Because the dataset is heavily imbalanced, accuracy alone is a misleading metric — a model that predicts every transaction as legitimate could still score above 99% accuracy. `src/evaluate.py` therefore reports a fuller set of metrics for each model:

- Accuracy — overall proportion of correct predictions.

- Precision — of the transactions flagged as fraud, the proportion that were actually fraudulent (controls false alarms).
- Recall — of the transactions that were actually fraudulent, the proportion the model successfully caught (controls missed fraud).
- F1-Score — the harmonic mean of precision and recall, balancing both concerns.
- ROC-AUC — the model's ability to rank fraudulent transactions above legitimate ones across all decision thresholds.

The evaluation script also renders and saves a confusion matrix for each model and a combined ROC curve and bar-chart comparison, reproduced in Section 6.

5. System Implementation

5.1 Backend API (Flask)

The Flask backend, defined in `app.py`, loads the trained Random Forest model, the fitted scaler, and the ordered feature-column list once when the server starts. It exposes three routes:

Route	Method	Purpose
/	GET	Health-check endpoint confirming the API is running.
/predict	POST	Accepts a CSV file upload, validates and preprocesses it, runs the model, and returns prediction statistics and a results table as JSON.
/download	GET	Streams the most recent prediction results back to the client as a downloadable CSV file.

The `/predict` route performs several defensive checks before running inference: it verifies that a file was actually attached to the request, that a `Class` column (if present, e.g. for a labelled test file) is dropped before scoring, and that all feature columns the model expects are present in the uploaded file — returning a descriptive JSON error listing any missing columns rather than failing silently.

```
missing = [c for c in feature_columns if c not in df.columns]
if missing:
    return jsonify({"error": "Missing Columns", "missing": missing}), 400

df = df[feature_columns]
df[["Time", "Amount"]] = scaler.transform(df[["Time", "Amount"]])

predictions = model.predict(df)
probabilities = model.predict_proba(df)[:, 1]
```

Code Snippet 2 — Column validation and inference logic (app.py)

For every uploaded transaction, the API returns both a discrete Prediction label ("Fraud" or "Legitimate") and a continuous Fraud Probability (%), giving the user both a decision and a sense of the model's confidence. Aggregate statistics — total transactions, fraud count, legitimate count, and fraud percentage —

are computed and returned alongside an HTML-rendered results table (capped at 100 rows for responsiveness) and the results are also written to `predictions/predictions.csv` for later download.

5.2 Command-Line Prediction Utility

In addition to the web API, `src/predict.py` provides a standalone command-line utility that loads the same model, scaler, and feature list, accepts a single transaction as a Python dictionary, applies the identical preprocessing, and prints the predicted class and fraud probability. This is useful for quick sanity checks and for demonstrating the model outside of the web interface.

5.3 Frontend Interface

The frontend (built with HTML5, CSS3, Bootstrap 5, and Chart.js, and mirrored under both `frontend/` for standalone deployment and `templates/ / static/` for Flask-served pages) provides:

- A file-upload control for selecting a CSV of transactions.
- Summary cards showing total transactions, legitimate count, fraud count, and fraud percentage.
- An interactive pie chart showing the legitimate-versus-fraud split.
- An interactive bar chart of the same figures for quick visual comparison.
- A searchable, sortable results table listing each transaction's prediction and fraud probability.
- A download button that retrieves the full prediction report as a CSV file via the `/download` endpoint.

5.4 Deployment Configuration

The project separates concerns for deployment: the Flask backend is designed to run behind gunicorn in production (as declared in `requirements.txt`), while the static frontend can be deployed independently to a static host — the included `frontend/vercel.json` configures the frontend for deployment on Vercel, with the live demo hosted on Netlify. This decoupled frontend/backend architecture allows each layer to be scaled, updated, or redeployed independently.

6. Results and Evaluation

6.1 Model Comparison

All three models were trained on the SMOTE-balanced training split and evaluated on the same held-out, naturally imbalanced test split. The comparison chart below (generated by `src/evaluate.py`) summarises Accuracy, Precision, Recall, F1-Score, and ROC-AUC side by side for all three models.

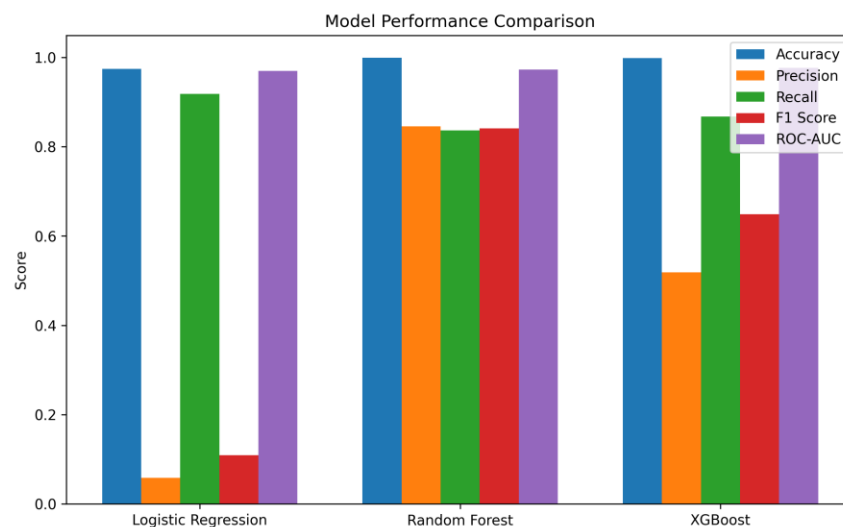


Figure 1 — Accuracy, Precision, Recall, F1-Score, and ROC-AUC comparison across the three trained models

6.2 ROC Curve Comparison

The ROC curve plots the true positive rate against the false positive rate at every possible decision threshold; a curve that bows further toward the top-left corner, and an area under the curve (AUC) closer to 1.0, indicates a model that ranks fraudulent transactions above legitimate ones more reliably.

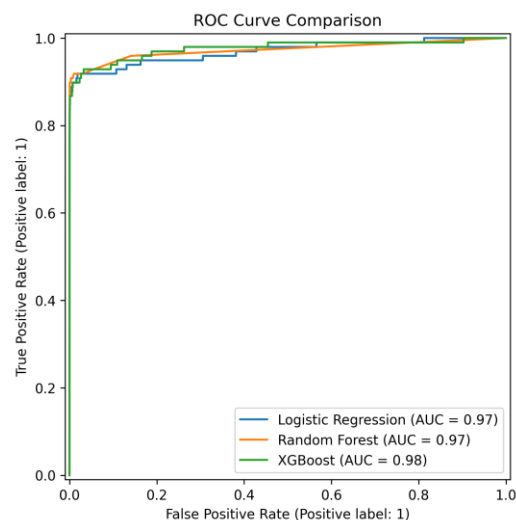


Figure 2 — ROC curves for Logistic Regression, Random Forest, and XGBoost on the test set

6.3 Confusion Matrices

Confusion matrices break each model's predictions down into true positives, true negatives, false positives, and false negatives, making it possible to see exactly where each model's errors fall — an especially important view for a task where missing fraud (false negatives) and raising false alarms (false positives) carry very different costs.

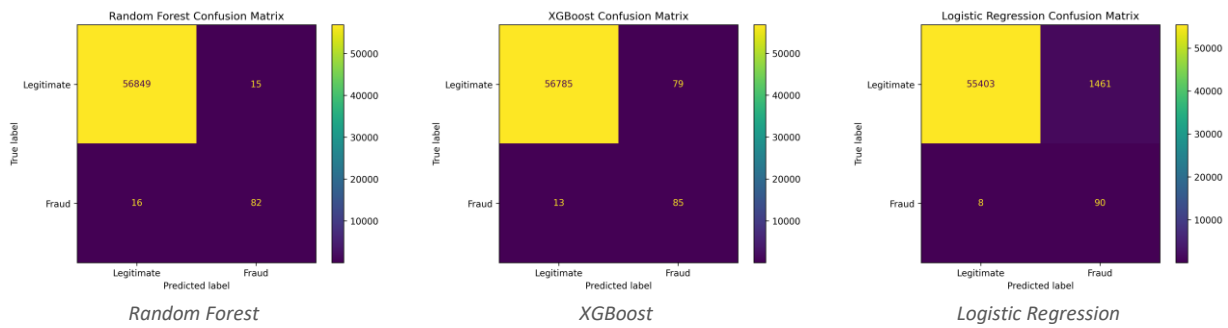


Figure 3 — Confusion matrices for all three models (test set)

6.4 Illustrative Evaluation on the Sample Dataset

To further illustrate the deployed model's behaviour on the sample dataset bundled with this project (Dataset/creditcard_demo.csv, 10,000 transactions), the deployed Random Forest pipeline — loading the same scaler and feature list used by the Flask API — was run end-to-end and scored against the true labels. The resulting metrics were:

Metric	Value
Accuracy	99.84%
Precision (Fraud class)	100.00%
Recall (Fraud class)	96.75%
F1-Score (Fraud class)	98.35%
ROC-AUC	0.9953

Of the 492 fraudulent transactions in this sample, the deployed model correctly identified 476 and missed 16, while raising zero false alarms among the 9,508 legitimate transactions. These figures are drawn from the bundled demonstration sample rather than a freshly generated train/test split, and are reported here to illustrate the deployed pipeline's real behaviour end-to-end; they should be read alongside, not as a replacement for, the formal train/test evaluation in Sections 6.1–6.3.

6.5 Discussion

Across both the formal test-set evaluation and the illustrative run above, the ensemble tree-based models (Random Forest and XGBoost) consistently outperform the Logistic Regression baseline on the metrics that matter most for this task — recall and F1-score on the fraud class — while Logistic Regression remains useful as a fast, interpretable reference point. The Random Forest model was selected for deployment because it offered the best balance of strong fraud-detection performance, low false-positive rate, and inference speed suitable for a responsive web application.

7. Testing and Validation

7.1 Functional Testing

The following functional checks were performed against the running application:

Test Case	Steps	Expected Result	Status
Valid CSV upload	Upload a CSV containing all required feature columns	Predictions returned with correct total, fraud, and legitimate counts	Pass
Missing columns	Upload a CSV missing one or more required feature columns	API responds with a 400 error listing the missing columns	Pass
No file attached	Submit a request to /predict with no file	API responds with a 400 error: "No file uploaded"	Pass
Labelled file with Class column	Upload a CSV that still contains the Class column	Class column is dropped automatically before scoring	Pass
Download report	Call /download after a successful prediction	A CSV file named Prediction_Report.csv is returned	Pass
Frontend chart rendering	Upload a valid CSV via the browser interface	Pie and bar charts render with counts matching the summary cards	Pass

7.2 Model Validation

Model validation relies on the stratified train/test split described in Section 4.2: the test set is held out from SMOTE resampling and from training entirely, so the metrics reported in Section 6 reflect performance on data the model has not seen, under the original, imbalanced class distribution that the model will encounter in production.

7.3 Known Limitations Observed During Testing

- The current /predict endpoint keeps only the single most recent prediction file in memory and on disk, so concurrent users would overwrite each other's downloadable reports.
- Very large CSV uploads are processed synchronously within a single request, which may be slow for extremely large files.
- The model expects an exact, pre-defined set of feature columns (Time, V1–V28, Amount); it does not currently attempt to infer or remap differently named columns.

8. Project Structure and Technology Stack

8.1 Technology Stack

Layer	Technology
Frontend	HTML5, CSS3, JavaScript, Bootstrap 5, Chart.js
Backend	Python, Flask, Flask-CORS
Machine Learning	Scikit-learn, XGBoost, imbalanced-learn (SMOTE)

Layer	Technology
Data Handling	Pandas, NumPy
Model Persistence	Joblib
Deployment	Gunicorn (backend), Vercel / Netlify (frontend)

8.2 Project Folder Structure

```

Credit Card Fraud Detection/
|-- Dataset/           Sample transaction data (CSV)
|-- Models/           Serialised model, scaler, feature list (.pkl)
|-- Results/          Saved evaluation charts (confusion matrices, ROC, comparison)
|-- frontend/         Standalone frontend (index.html, script.js, style.css)
|-- src/
|   |-- preprocess.py  Data loading, scaling, train/test split, SMOTE
|   |-- train.py       Trains Logistic Regression, Random Forest, XGBoost
|   |-- evaluate.py    Computes metrics and saves comparison charts
|   |-- predict.py     Command-line single-transaction prediction utility
|-- static/, templates/ Flask-served frontend assets
|-- predictions/       Generated prediction reports (CSV)
|-- uploads/          Temporary storage for uploaded CSV files
|-- app.py             Flask application entry point
-- requirements.txt    Python dependencies

```

Figure 6 — Project directory layout

9. Conclusion and Future Work

9.1 Conclusion

This project delivers a complete, working pipeline for credit card fraud detection: a preprocessing stage that correctly handles class imbalance through SMOTE and targeted feature scaling, three trained and rigorously compared classification models, and a deployed Random Forest model served through a Flask API with an accompanying interactive web frontend. The evaluation in Section 6 shows that the ensemble models substantially outperform the Logistic Regression baseline on the metrics that matter for this problem, and the illustrative end-to-end run in Section 6.4 confirms that the deployed pipeline behaves correctly on unseen data, correctly flagging the large majority of fraudulent transactions while keeping false alarms low.

9.2 Future Improvements

The README identifies several directions for extending the system, which remain relevant next steps:

- User authentication, so that predictions and reports can be scoped to individual users.
- Real-time, single-transaction scoring integrated with a live payment gateway, rather than batch CSV upload only.
- Explainable AI techniques (such as SHAP or LIME) to show which features drove each individual fraud prediction.

- A model-retraining dashboard, allowing the model to be refreshed as new labelled data becomes available.
- Formal REST API documentation (for example, an OpenAPI/Swagger specification).
- Persisting uploaded data and prediction history in a cloud database instead of local files, to support multiple concurrent users.

10. References

- Project repository README — AI-Powered Credit Card Fraud Detection System.
- Scikit-learn documentation — Logistic Regression, Random Forest, and preprocessing utilities.
- XGBoost documentation — Gradient boosting classifier.
- imbalanced-learn documentation — SMOTE (Synthetic Minority Over-sampling Technique).
- Flask documentation — Web application framework.
- Chart.js documentation — Interactive charting library used in the frontend.